

GROKING: GENERALIZATION BEYOND OVERFITTING ON SMALL ALGORITHMIC DATASETS

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin
OpenAI

Vedant Misra*
Google

ABSTRACT

In this paper we propose to study generalization of neural networks on small algorithmically generated datasets. In this setting, questions about data efficiency, memorization, generalization, and speed of learning can be studied in great detail. In some situations we show that neural networks learn through a process of “grokking” a pattern in the data, improving generalization performance from random chance level to perfect generalization, and that this improvement in generalization can happen well past the point of overfitting. We also study generalization as a function of dataset size and find that smaller datasets require increasing amounts of optimization for generalization. We argue that these datasets provide a fertile ground for studying a poorly understood aspect of deep learning: generalization of overparametrized neural networks beyond memorization of the finite training dataset.

1 INTRODUCTION

The generalization of overparameterized neural networks has long been a source of interest to the machine learning community since it defies intuitions derived from classical learning theory. In this paper we show that training networks on small algorithmically generated datasets can reliably exhibit unusual generalization patterns, clearly decoupled from performance on the training set, in a significantly more pronounced way than such effects manifest on datasets derived from natural data (see Figure 1, left, for an example). Such experiments can be quickly reproduced on a single GPU, and this makes them convenient testbeds for theories of generalization.

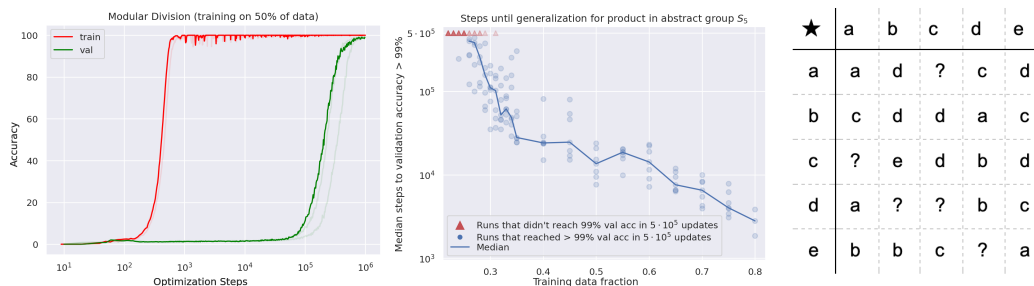


Figure 1: **Left.** Grokking: A dramatic example of generalization far after overfitting on an algorithmic dataset. We train on the binary operation of division mod 97 with 50% of the data in the training set. Each of the 97 residues is presented to the network as a separate symbol, similar to the representation in the figure to the right. The red curves show training accuracy and the green ones show validation accuracy. Training accuracy becomes close to perfect at $< 10^3$ optimization steps, but it takes close to 10^6 steps for validation accuracy to reach that level, and we see very little evidence of any generalization until 10^5 steps. **Center.** Training time required to reach 99% validation accuracy increases rapidly as the training data fraction decreases. **Right.** An example of a small binary operation table. We invite the reader to make their guesses as to which elements are missing.

*Vedant was at OpenAI at the time of this work

The datasets we consider are binary operation tables of the form $a \circ b = c$ where a, b, c are discrete symbols with no internal structure, and $\circ$ is a binary operation. Examples of binary operations include addition, composition of permutations, and bivariate polynomials. Training a neural network on a proper subset of all possible equations then amounts to filling in the blanks of the binary op table, much like solving a Sudoku puzzle. An example is shown on the right in Figure 1. Since we use distinct abstract symbols for all distinct elements a, b, c involved in the equations, the network is not made aware of any internal structure of the elements, and has to learn about their properties only from their interactions with other elements. For example the network doesn’t see numbers in decimal notation, or permutations in line notation.

Our contributions are as follows:

- We show that neural networks are capable of generalizing to the empty slots in a variety of binary op tables.
- We show that, long after severely overfitting, validation accuracy sometimes suddenly begins to increase from chance level toward perfect generalization. We call this phenomenon ‘grokking’. An example is shown in Figure 1.
- We present the data efficiency curves for a variety of binary operations.
- We show empirically that the amount of optimization required for generalization quickly increases as the dataset size decreases.
- We compare various optimization details to measure their impact on data efficiency. We find that weight decay is particularly effective at improving generalization on the tasks we study.
- We visualize the symbol embeddings learned by these networks and find that they sometimes uncover recognizable structure of the mathematical objects represented by the symbols.

2 METHOD

All of our experiments used a small transformer trained on datasets of equations of the form $a \circ b = c$, where each of “ a ”, “ $\circ$ ”, “ b ”, “ $=$ ”, and “ c ” is a separate token. Details of the operations studied, the architecture, training hyperparameters and tokenization can be found in Appendix A.1.

3 EXPERIMENTS

3.1 GENERALIZATION BEYOND OVERFITTING

Deep learning practitioners are used to seeing small improvements in validation accuracy after validation loss stops decreasing. A double descent of validation loss has been documented in some circumstances, but is considered unusual among practitioners Nakkiran et al. (2019); Belkin et al. (2018); d’Ascoli et al. (2020). On the small algorithmic datasets that we study, improved generalization after initial overfitting occurs for a range of models, optimizers, and dataset sizes, and in some cases these effects are extremely pronounced. A typical example is shown for modular division in Figure 1. There we see that validation accuracy starts increasing beyond chance level only after 1000 times more optimization steps than are required for training accuracy to get close to optimal. In Figure 4 the training/validation losses are also plotted and we see the double descent of the validation loss.

We found these behaviors to be typical for all the binary operations for dataset sizes that were close to the minimal dataset size for which the network generalized within the allotted optimization budget. For larger dataset sizes, the training and validation curves tend to track each other more closely.

3.1.1 LEARNING TIME CURVES

In a typical supervised learning problem, decreasing the amount of training data decreases the converged generalization performance of the model when the optimization procedure is capable of interpolating the training data. In our setting, we observe a different phenomenon: while the converged performance stays constant at 100% within a range of training dataset sizes, the optimization time required to achieve that performance grows quickly as the dataset size is decreased.

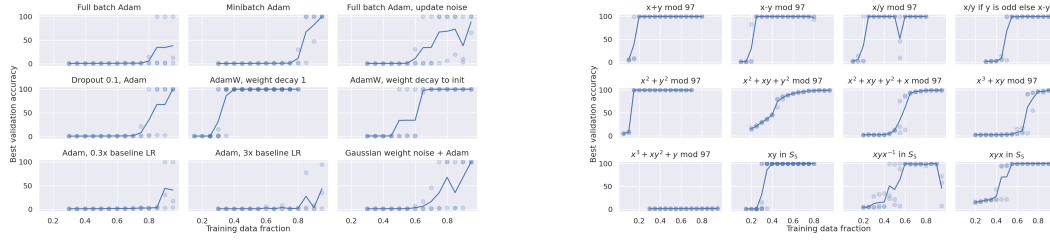


Figure 2: **Left.** Different optimization algorithms lead to different amounts of generalization within an optimization budget of 10^5 steps for the problem of learning the product in the abstract group S_5 . Weight decay improves generalization the most, but some generalization happens even with full batch optimizers and models without weight or activation noise at high percentages of training data. Suboptimal choice hyperparameters severely limit generalization. Not shown: training accuracy reaches 100% after 10^3 - 10^4 updates for all optimization methods. **Right.** Best validation accuracy achieved after 10^5 steps on a variety of algorithmic datasets, averaged over 3 seeds. Generalization happens at higher percentages of data for intuitively more complicated and less symmetrical operations.

Figure 1 (center) shows median number of optimization steps until validation performance first reaches 99% for the product in abstract group S_5 . In the vicinity of 25-30% of data, a decrease of 1% of training data leads to an increase of 40-50% in median time to generalization. While the number of steps until validation accuracy $> 99\%$ grows quickly as dataset size decreases, the number of steps until the train accuracy first reaches 99% generally trends down as dataset size decreases and stays in the range of 10^3 - 10^4 optimization steps. We’ve observed a similar pattern of exponential increase in optimization time until reaching generalization as dataset size decreases on all the algorithmic tasks for which we could get the networks to generalize.

3.2 GROKING ON A VARIETY OF PROBLEMS

We’ve measured the mean accuracy across three runs for training datasets consisting of different fractions of all available equations for a variety of binary operations listed in Appendix A.1.1. The results are presented in Figure 2 (right).

Since the operands are presented to the neural network as unrelated abstract symbols, the operations $x + y \pmod{p-1}$ and $x * y \pmod{p}$ with a prime number p and non-zero x, y are indistinguishable from the neural network’s perspective (and similarly $x - y \pmod{p-1}$ and $x/y \pmod{p}$). This is because every nonzero residue modulo a prime can be represented as a power of a primitive root. This representation shows the equivalence (up to renaming of symbols) of modular addition modulo $p-1$ and modular multiplication modulo p . We see in Figure 2 (right) that $x - y$ and x/y indeed take about the same amount of data for generalization to occur.

Some of the operations listed in Figure 2 (right) are symmetric with respect to the order of the operands ($x + y$, $x * y$, $x^2 + y^2$ and $x^2 + xy + y^2$). Such operations tend to require less data for generalization than closely related non-symmetrical counterparts ($x - y$, x/y , $x^2 + xy + y^2 + x$). We believe this effect might be partially architecture-dependent, since it’s easy for a transformer to learn a symmetric function of the operands by ignoring positional embedding.

Some operations (for example $x^3 + xy^2 + y \pmod{97}$) didn’t lead to generalization within the allowed optimization budget at any percentage of data up to 95%. The converged models effectively just memorized the training dataset without finding any real patterns in the data. To such a model, the data is effectively random.

The operation $[x/y \pmod{p}]$ if y is odd, otherwise $x - y \pmod{p}$ requires the network to learn a mix of several simple operations - in particular the role of x has to be interpreted as a residue in the additive group when it’s paired with an even y , and as a residue in the multiplicative group when it’s paired with an odd y . This shows that generalization can happen even for operations that are not cleanly interpretable via group or ring operations.

3.3 ABLATIONS AND TRICKS

We’ve tried various forms of regularization to see what can induce networks to generalize better on our datasets. Here we present the data efficiency curves on a particular dataset S_5 for a variety of interventions: full-batch gradient descent, stochastic gradient descent, large or small learning rates,